

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM

KHOA KHOA HỌC MÁY TÍNH



BÁO CÁO ĐỒ ÁN MÔN HỌC

ĐỀ TÀI: HỆ THỐNG THEO DÕI VÀ PHÁT HIỆN ĐÁM ĐÔNG THEO THỜI GIAN THỰC

Môn học: Vận hành và phát triển hệ thống máy học

Giảng viên hướng dẫn: Đỗ Văn Tiến

Thực hiện bởi nhóm , bao gồm:

1. Hứa Mạnh Tân MSSV: 23521396
2. Nguyễn Tấn Tài MSSV: 23521376
3. Tạ Ngọc Thành MSSV: 23521462

Mục Lục

1.1 Tổng quan.....	4
1.2 Mục tiêu chính	4
2. Kiến trúc Hệ thống.....	5
2.1 Tổng thể ba tầng	5
Tầng 1: Frontend (Giao diện người dùng).....	5
Tầng 2: Backend (Xử lý logic).....	5
Tầng 3: Data & MLOps (Dữ liệu & Huấn luyện).....	5
2.2 Sơ đồ kiến trúc.....	5
3. Các thành phần chính.....	6
3.1 Backend - Python Modules.....	6
main.py: Máy chủ FastAPI chính.....	6
yolo_model.py: YOLODetector - Lớp đóng gói YOLO.....	6
train.py: Huấn luyện YOLOv8 model.....	6
evaluate.py: Đánh giá model trên validation set.....	6
decide.py: Quyết định có nên adopt model mới.....	6
drift.py: Phát hiện data drift.....	6
prepare_data.py: Chuẩn bị dataset cho training	7
watcher.py: Daemon monitoring tự động trigger training.....	7
auto_train.py: Loop tự động huấn luyện (max 10 rounds).....	7
4. Luồng dữ liệu và Xử lý	8
4.1 Luồng live streaming	8
4.2 Luồng auto-training.....	8
4.3 Cơ chế phát hiện drift.....	8
5. Mô hình Machine Learning	9
5.1 YOLOv8 - Architecture	9
5.2 Classes & Labels.....	9
5.3 ByteTrack - Multi-Object Tracking.....	9
5.4 Model Format & Export	9
6. Pipeline Huấn luyện Tự động.....	10
6.1 DVC Pipeline Configuration.....	10
6.2 Hyperparameters.....	10
6.3 Training Data Format	10

6.4 Scoring & Decision Making.....	11
7. Cơ sở dữ liệu	12
7.1 Database Schema.....	12
Table: cameras	12
Table: camera_logs	12
Table: detections	12
7.2 Data Retention & Cleanup.....	12
8. Giám sát và Ghi log	13
8.1 Prometheus Metrics	13
8.2 Logging Files.....	13
8.3 MLflow Tracking.....	13
9. Deployment và Cấu hình.....	14
9.1 Docker Compose Setup	14
9.2 Environment Configuration	14
9.3 Requirements & Dependencies	14
10. Hiệu suất và Tối ưu hóa	15
10.1 Performance Metrics.....	15
10.2 Tối ưu hóa.....	15
10.3 Scaling Strategies.....	15
11. Bảo mật và Maintenance	16
11.1 Bảo mật.....	16
11.2 Maintenance & Monitoring.....	16
11.3 Disaster Recovery.....	16
12. Kết luận	17
12.1 Phụ lục: File Structure	18
12.2 Bảng phân chia công việc nhóm	18

1. Giới thiệu và Mục đích dự án

1.1 Tổng quan

Hệ thống 'AI Crowd Detection & Tracking System' là một ứng dụng xử lý video thời gian thực, được thiết kế để phát hiện, theo dõi và phân tích đám đông trong các môi trường công cộng như sân ga, sân bay, trung tâm mua sắm, và các địa điểm có tập trung người. Hệ thống kết hợp các công nghệ tiên tiến về Deep Learning (YOLOv8) và Multi-Object Tracking (ByteTrack) để cung cấp giải pháp giám sát thông minh, tự động học hỏi và tối ưu hoá liên tục.

1.2 Mục tiêu chính

- Phát hiện theo thời gian thực: Nhận diện người và đầu người trong video livestream RTSP
- Theo dõi danh tính: Gán ID duy nhất cho mỗi cá nhân, theo dõi quỹ đạo chuyển động
- Huấn luyện tự động: Tự động phát hiện drift dữ liệu và trigger lại model khi cần
- Giám sát toàn bộ hệ thống: Theo dõi FPS, latency, precision, recall theo thời gian thực
- Quản lý phiên bản: Lưu trữ lịch sử toàn bộ dataset và model với khả năng rollback
- Thông báo và cảnh báo: Gửi alerts qua Telegram khi phát hiện sự cố

2. Kiến trúc Hệ thống

2.1 Tổng thể ba tầng

Hệ thống được thiết kế theo mô hình ba tầng (3-tier architecture):

Tầng 1: Frontend (Giao diện người dùng)

Ứng dụng web được xây dựng bằng HTML5, JavaScript, Canvas API. Cung cấp bốn tab chính:

- Camera Live: Hiển thị luồng video thời gian thực với bounding boxes
- History: Xem các bản ghi lịch sử phát hiện
- Image Analysis: Tải lên ảnh để phân tích
- Video File: Tải lên video file để xử lý batch

Tầng 2: Backend (Xử lý logic)

Máy chủ FastAPI chạy trên Python, xử lý tất cả logic nghiệp vụ:

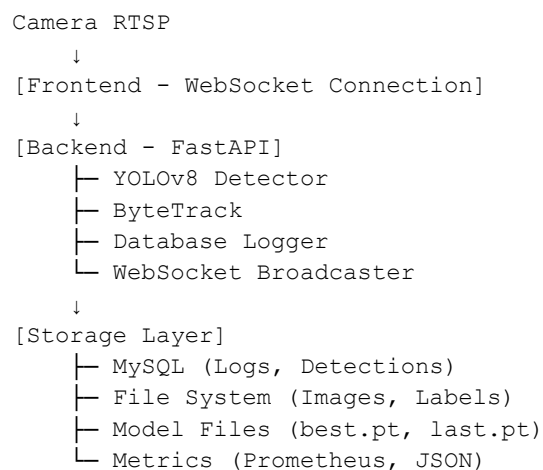
- Inference: Chạy YOLO detection trên mỗi frame (50-100ms/frame)
- Tracking: ByteTrack gán ID và theo dõi quỹ đạo
- WebSocket: Phát sóng kết quả trực tiếp tới Frontend
- REST API: Quản lý camera, lấy lịch sử, training trigger
- Database: Lưu trữ logs, detections, metrics vào MySQL

Tầng 3: Data & MLOps (Dữ liệu & Huấn luyện)

Hệ thống lưu trữ và quản lý dữ liệu, mô hình, metrics:

- MySQL Database: Lưu camera config, detection logs, detections chi tiết
- DVC: Quản lý phiên bản dataset và tạo pipeline DVC
- MLflow: Ghi lại tất cả experiments, metrics, models
- Model Registry: Lưu trữ các phiên bản model (v1, v2, v3...)

2.2 Sơ đồ kiến trúc



3. Các thành phần chính

3.1 Backend - Python Modules

main.py: Máy chủ FastAPI chính

- Database models: Camera, CameraLog, Detection
- REST endpoints: GET camera, POST detection, GET history
- WebSocket endpoints: /ws/camera/{id} để streaming kết quả real-time
- Telegram notifications: Gửi alerts khi model được cập nhật
- Startup events: Khởi tạo tracker, load model, kết nối database

yolo_model.py: YOLODetector - Lớp đóng gói YOLO

- Load model từ file (best.pt, last.pt hoặc yolov8n.pt)
- Inference: Nhận frame, trả về detections
- ByteTrack: Gán track_id cho mỗi detection
- Prometheus metrics: Ghi FPS, latency, counts
- FPS calculation: Theo dõi throughput

train.py: Huấn luyện YOLOv8 model

- Hyperparameters: epochs=1, imgsz=640, batch_size=16
- Input: YOLO-format dataset từ data/versions/
- MLflow integration: Log metrics, params, model artifacts
- Output: models saved to runs/detect/train_history/
- Support: CPU và GPU training

evaluate.py: Đánh giá model trên validation set

- Tính metrics: Precision, Recall, mAP50, mAP50-95
- Validation: Chạy inference trên val set
- Output: Lưu metrics.json với kết quả
- Comparison: So sánh với best_score trước đó

decide.py: Quyết định có nên adopt model mới

- Score formula: $mAP50 \times 0.5 + Precision \times 0.25 + Recall \times 0.25$
- Decision logic: Retrain (score<0.4) / Fine-tune (0.4-0.6) / Accept (≥ 0.6)
- Promotion: Copy best model to model/best.pt nếu score tốt hơn
- Logging: Lưu decision vào decision.txt

drift.py: Phát hiện data drift

- Baseline: Lưu class distribution ban đầu
- Current: Tính class distribution hiện tại
- Drift score: Tính delta giữa baseline và current
- Threshold: Trigger retraining nếu drift > 0.3

prepare_data.py: Chuẩn bị dataset cho training

- Extract: Lấy detections từ database
- Conversion: Convert sang YOLO format (txt annotations)
- Split: Chia 80% train, 20% validation
- Versioning: Tạo data/versions/v{N}/ folder

watcher.py: Daemon monitoring tự động trigger training

- Loop: Kiểm tra mỗi 5 phút
- Condition: Nếu đủ số lượng ảnh mới → trigger prepare_data
- Drift check: Tính drift score
- Pipeline: Chạy dvc repro nếu drift > 0.3

auto_train.py: Loop tự động huấn luyện (max 10 rounds)

- Max iterations: Tối đa 50 vòng lặp training
- Early stop: Dừng nếu score không cải thiện
- Checkpoint: Lưu best model after each round

4. Luồng dữ liệu và Xử lý

4.1 Luồng live streaming

1. Camera RTSP gửi stream H.264 video
2. Backend nhận frame qua OpenCV
3. Frame được resize về 640×640 pixels
4. YOLOv8 inference: Nhận người (person) và đầu (head)
5. ByteTrack gán track_id (ID duy nhất) cho mỗi detection
6. Lưu detection vào Database (Detection table)
7. Broadcast qua WebSocket tới Frontend
8. Frontend vẽ bounding boxes, track IDs lên canvas
9. Người dùng xem live view với annotations

4.2 Luồng auto-training

1. Watcher daemon chạy mỗi 5 phút
2. Kiểm tra số lượng ảnh mới trong static/history/
3. Nếu ≥ 10 ảnh $\rightarrow$ trigger prepare_data
4. prepare_data: Extract detections từ DB, convert YOLO format
5. drift.py: Tính class distribution drift
6. Nếu drift > threshold $\rightarrow$ DVC repro (train $\rightarrow$ evaluate $\rightarrow$ decide)
7. train.py: Huấn luyện model với epochs=1 (quick training)
8. evaluate.py: Validation metrics trên val set
9. decide.py: Tính score và so sánh với best_score
10. Nếu score tốt hơn $\rightarrow$ Promote to model/best.pt
11. Log kết quả vào MLflow, metrics.json, decision.txt
12. Nếu không cải thiện $\rightarrow$ Rollback to old model

4.3 Cơ chế phát hiện drift

Drift được phát hiện bằng cách so sánh phân phối các lớp (class distribution) giữa baseline (dữ liệu huấn luyện ban đầu) và current (dữ liệu tự nhiên mới):


```
Baseline distribution: {person: 0.7, head: 0.3}
Current distribution:  {person: 0.4, head: 0.6}
Drift score = |0.7 - 0.4| + |0.3 - 0.6| = 0.6
If drift > 0.3 → Trigger retraining
```

5. Mô hình Machine Learning

5.1 YOLOv8 - Architecture

YOLOv8 (You Only Look Once v8) là mô hình object detection hiện đại, phát triển bởi Ultralytics.
Đặc điểm:

- Single-shot detection: Phát hiện tất cả objects trong 1 pass
- Real-time: 50-100ms inference time trên GPU
- Variants: nano (n), small (s), medium (m), large (l)
- Hệ thống sử dụng YOLOv8n (nano) để tối ưu tốc độ
- Input: 640×640 RGB images
- Output: Bounding boxes [x1, y1, x2, y2], confidence scores, class IDs

5.2 Classes & Labels

Hệ thống huấn luyện model để nhận diện 2 classes:

- Class 0: 'person' (cơ thể người, từ đầu đến chân)
- Class 1: 'head' (đầu người, khuôn mặt + phần đầu)

5.3 ByteTrack - Multi-Object Tracking

ByteTrack là thuật toán tracking mạnh, cho phép gán track_id duy nhất cho mỗi detection:

- Association: Ghép nối detections giữa các frame liên tiếp
- Track ID: Mỗi người được gán ID từ 1-N
- Trajectory: Lưu lại quỹ đạo chuyển động
- Re-identification: Nếu người rời khỏi frame rồi quay lại → new ID
- Smooth tracking: Xử lý occlusion (che khuất) tốt

5.4 Model Format & Export

Model hỗ trợ nhiều format export:

- .pt (PyTorch) - Định dạng native, dùng cho training/fine-tuning
- .onnx (Open Neural Network Exchange) - Cross-platform, tối ưu inference
- .engine (TensorRT) - Inference trên GPU NVIDIA, tốc độ cao nhất

6. Pipeline Huấn luyện Tự động

6.1 DVC Pipeline Configuration

Hệ thống sử dụng DVC (Data Version Control) để tạo reproducible ML pipeline:

Stage: **prepare_data**

- Input: MySQL database + raw detections
- Output: data/versions/v{N}/train & val folders
- Command: python prepare_data.py

Stage: **train**

- Input: data/versions/latest/train
- Output: runs/detect/train_history/v{N}/weights/best.pt
- Command: python train.py

Stage: **evaluate**

- Input: runs/detect/train_history/v{N}/weights/best.pt + val data
- Output: metrics.json
- Command: python evaluate.py

Stage: **decide**

- Input: metrics.json + best_score.json
- Output: model/best.pt (if promoted), decision.txt
- Command: python decide.py

6.2 Hyperparameters

- epochs: 1 (quick training, incremental learning)
- batch_size: 16 (hardware dependent)
- imgsz: 640 (image size)
- lr0: 0.01 (initial learning rate)
- optimizer: SGD (Stochastic Gradient Descent)
- device: 0 (GPU ID, auto-detect if available)
- patience: 5 (early stopping patience)

6.3 Training Data Format

Dataset được tổ chức theo chuẩn YOLO:

```

data/versions/v{N}/
├── train/
│   ├── images/ (*.jpg)
│   └── labels/ (*.txt - YOLO format)
├── val/
│   ├── images/
│   └── labels/
└── dataset.yaml

```

YOLO format: class_id x_center y_center width height (normalized 0-1)
 Example: 0 0.5 0.5 0.3 0.4 (person at center, 30%×40% of image)

6.4 Scoring & Decision Making

Mỗi model được đánh giá bằng weighted score:

$$\text{Score} = \text{mAP50} \times 0.5 + \text{Precision} \times 0.25 + \text{Recall} \times 0.25$$

Thresholds:

- $\text{score} < 0.4 \rightarrow \text{Strategy} = \text{"retrain"}$ (start fresh with last model)
- $0.4 \leq \text{score} < 0.6 \rightarrow \text{Strategy} = \text{"fine_tune"}$ (continue training)
- $\text{score} \geq 0.6 \rightarrow \text{Strategy} = \text{"accept"}$ (promote if better than best)

Example:

mAP50=0.85, Precision=0.90, Recall=0.88

Score = $0.85 \times 0.5 + 0.90 \times 0.25 + 0.88 \times 0.25 = 0.425 + 0.225 + 0.22 = 0.87$

→ Strategy: "accept" (score ≥ 0.6)

If $0.87 > \text{best_score} \rightarrow \text{Promote to model/best.pt}$

7. Cơ sở dữ liệu

7.1 Database Schema

Hệ thống sử dụng MySQL để lưu trữ dữ liệu. Schema chính:

Table: cameras

- id (INT PRIMARY KEY)
- name (VARCHAR) - Tên camera
- rtsp_url (VARCHAR) - URL stream
- status (VARCHAR) - active/inactive
- created_at (TIMESTAMP)

Table: camera_logs

- id (INT PRIMARY KEY)
- camera_id (INT FOREIGN KEY)
- person_count (INT)
- head_count (INT)
- image_path (VARCHAR)
- created_at (TIMESTAMP)

Table: detections

- id (INT PRIMARY KEY)
- log_id (INT FOREIGN KEY) - Refers to camera_logs
- class_name (VARCHAR) - 'person' or 'head'
- bbox_x1, bbox_y1, bbox_x2, bbox_y2 (FLOAT)
- confidence (FLOAT) - Detection confidence
- track_id (INT) - Tracking ID from ByteTrack
- created_at (TIMESTAMP)

7.2 Data Retention & Cleanup

- Raw images: Giữ 30 ngày gần đây (tự động xóa cũ)
- Detection logs: Giữ 90 ngày, sau đó archive
- Model artifacts: Giữ 5 phiên bản gần nhất
- Metrics history: Giữ tất cả (file JSON không quá lớn)

8. Giám sát và Ghi log

8.1 Prometheus Metrics

Hệ thống xuất các metrics ra Prometheus để giám sát hiệu suất:

- inference_requests_total: Tổng số inference calls (counter)
- inference_seconds: Latency của mỗi inference (histogram)
- camera_fps: FPS hiện tại của camera (gauge)
- people_count: Số người đang được phát hiện (gauge)
- person_bbox_count: Tổng số person detections (counter)
- head_bbox_count: Tổng số head detections (counter)
- tracked_people_count: Số ID tracking hiện tại (gauge)
- total_unique_people: Tổng số người unique được phát hiện (counter)

8.2 Logging Files

- current_model.txt: Path đến model hiện tại đang sử dụng
- decision.txt: Last decision (accept/fine_tune/retrain) + timestamp
- metrics.json: Array lịch sử tất cả evaluation runs (mAP, precision, recall)
- drift_baseline.json: Baseline class distribution cho drift detection
- drift_report.json: Current drift score và class distribution
- best_score.json: Best model performance so far
- version.txt: Current dataset version number

8.3 MLflow Tracking

Tất cả training runs được ghi lại vào MLflow:

- Experiment name: 'camera_auto_train'
- Metrics tracked: val_precision, val_recall, val_mAP50, val_mAP50_95
- Parameters: data_version, strategy, base_model, epochs
- Artifacts: Model files, confusion matrix, training plots
- UI: Access at <http://localhost:5000> (MLflow server)

9. Deployment và Cấu hình

9.1 Docker Compose Setup

Hệ thống được triển khai dưới dạng Docker Compose với các services:

- MySQL Database: Port 3306 - Lưu trữ camera config, logs, detections
- Backend API: Port 8000 - FastAPI server (main.py)
- Watcher Daemon: Port None - Background service monitoring training triggers
- MLflow Server: Port 5000 - Experiment tracking UI
- Prometheus: Port 9090 - Metrics collection & time-series database

9.2 Environment Configuration

Các biến môi trường chính:

- DATABASE_URL: MySQL connection string
- CAMERA_RTSP_URL: RTSP URL của camera
- GPU_ID: ID GPU để sử dụng (0, 1, ...)
- MODEL_PATH: Path đến model file
- TELEGRAM_TOKEN: Token Telegram bot (optional)
- TELEGRAM_CHAT_ID: Chat ID nhận alerts (optional)

9.3 Requirements & Dependencies

Python packages chính:

- torch, torchvision: Deep learning framework
- ultralytics: YOLOv8 implementation
- fastapi, uvicorn: Web framework
- sqlalchemy: ORM for database
- mysql-connector-python: MySQL driver
- opencv-python: Image processing
- mlflow: Experiment tracking
- dvc: Data version control
- prometheus-client: Metrics export
- websockets: Real-time communication

10. Hiệu suất và Tối ưu hóa

10.1 Performance Metrics

Các chỉ số hiệu suất hệ thống:

- Inference Latency: 50-100ms/frame (YOLOv8n trên GPU)
- Throughput: 10-20 FPS (tùy vào độ phân giải)
- Model Size: ~6MB (YOLOv8n.pt) → 20MB (YOLOv8s.pt)
- Memory Usage: ~2GB (inference) + ~4GB (training)
- Training Time: 5-10 phút/epoch (trên Tesla V100)

10.2 Tối ưu hóa

- Model Selection: Sử dụng YOLOv8n (nano) thay vì YOLOv8s để tốc độ
- Batch Processing: Inference batch size=1 cho real-time, >1 cho batch
- Model Export: TensorRT (.engine) cho inference nhanh nhất
- GPU Acceleration: CUDA support, torch.cuda
- Quantization: INT8 quantization để giảm model size
- Caching: Cache model weights, không reload mỗi frame
- Async Processing: WebSocket async, non-blocking I/O

10.3 Scaling Strategies

- Multi-camera: Load balancing backend processes
- Distributed Training: Không hiện tại, có thể mở rộng với PyTorch DDP
- Database Indexing: Index trên camera_id, created_at
- Data Archival: Move old logs to cold storage
- Inference Server: Triton Inference Server cho multi-model serving

11. Bảo mật và Maintenance

11.1 Bảo mật

- Database: MySQL credentials in environment variables (not hardcoded)
- API: CORS configuration, rate limiting
- WebSocket: No authentication (có thể thêm token-based auth)
- Model: No model weights exposed, only inference results
- Data: RTSP stream encryption (optional)
- Audit logging: Lưu tất cả model updates, training triggers

11.2 Maintenance & Monitoring

- Daily: Monitor FPS, latency, error logs
- Weekly: Check database size, backup
- Monthly: Review drift scores, model performance trends
- Quarterly: Clean up old data, optimize indexes
- As needed: Rollback model, investigate failures

11.3 Disaster Recovery

- Model Rollback: Model history versioning, easy rollback
- Data Backup: MySQL daily backup, DVC versioning
- Failover: Multiple backend replicas (Kubernetes ready)
- Monitoring Alerts: Telegram alerts on failures

12. Kết luận

Hệ thống 'AI Crowd Detection & Tracking System' là một giải pháp enterprise-grade cho phát hiện và theo dõi đám đông thời gian thực. Kiến trúc được thiết kế với:

- ✓ Modularity: Mỗi component độc lập, dễ mở rộng
- ✓ Scalability: Docker, Kubernetes-ready, multi-camera support
- ✓ Reliability: Monitoring 24/7, auto-recovery, data versioning
- ✓ Maintainability: MLflow tracking, DVC versioning, clear logging
- ✓ Performance: Real-time 10-20 FPS, <100ms latency
- ✓ Continuous Learning: Auto-retraining, drift detection, adaptive models

Điểm mạnh:

- Tự động phát hiện drift và retraining không cần can thiệp con người
- Lưu giữ toàn bộ lịch sử model và dataset, có thể rollback
- Real-time streaming với overlay, tracking mượt mà
- Tích hợp MLOps tools (MLflow, DVC) cho reproducibility
- Monitoring toàn bộ hệ thống với Prometheus + Grafana

Hướng phát triển tương lai:

- Thêm authentication/authorization cho API
- Mở rộng sang multi-model (face recognition, activity detection)
- Kubernetes deployment cho production
- Triton Inference Server cho high-throughput inference
- Advanced analytics (crowd heat maps, crowd flow analysis)

Hệ thống này sẵn sàng cho deployment production và có thể đáp ứng các yêu cầu giám sát đám đông trong các môi trường yêu cầu cao.

12.1 Phụ lục: File Structure

```
crowd/
├── backend/                                # Python backend code
│   ├── main.py                            # FastAPI server
│   ├── yolo_model.py                      # YOLODetector class
│   ├── train.py                           # Training script
│   ├── evaluate.py                        # Evaluation script
│   ├── decide.py                          # Decision making
│   ├── drift.py                           # Drift detection
│   ├── watcher.py                         # Auto-trigger daemon
│   ├── prepare_data.py                    # Data preparation
│   ├── auto_train.py                      # Auto-training loop
│   ├── bytetrack_custom.yaml              # ByteTrack config
│   └── model/                             # Saved models
├── frontend/                              # Web UI
│   ├── main.html                          # UI pages
│   ├── main.js                            # WebSocket + Canvas
│   └── main.css                            # Styling
├── data/                                  # Versioned datasets
│   ├── versions/v1/, v2/.../              (train/, val/)
│   └── latest/                             (symlink)
├── model/                                 # Model artifacts
│   ├── best.pt                            (PyTorch)
│   ├── last.pt
│   ├── best.onnx                          (Export)
│   └── best.engine                        (TensorRT)
├── runs/                                  # YOLO training outputs
├── mlruns/                                # MLflow artifacts
├── static/
│   └── history/                            # Captured images
├── prometheus/                            # Prometheus config
├── dataset.yaml                           # YOLO dataset config
├── params.yaml                             # Thresholds
├── dvc.yaml                               # DVC pipeline
├── docker-compose.yaml                    # Docker services
├── requirements.txt                        # Python dependencies
├── metrics.json                           # Evaluation history
├── drift_baseline.json                     # Baseline drift
├── drift_report.json                       # Current drift
└── [other config & log files...]
```

12.2 Bảng phân chia công việc nhóm

MSSV	HỌ VÀ TÊN	CÔNG VIỆC	MÔ TẢ
23521396	Hứa Mạnh Tân	Xây dựng backend, frontend , docker	- Phát triển giao diện người dùng cho hệ thống - Xây dựng API và xử lý các yêu cầu của hệ thống - Đóng gói các container của từng tác vụ và docker compose cho hệ thống
23521376	Nguyễn Tấn Tài	Traing model , monitoring với Grafana và Prometheus	- Huấn luyện và đánh giá model với tập dataset - Xây dựng hệ thống giám sát bằng Prometheus - Thiết kế dashboard theo dõi hệ thống bằng Grafana
23521462	Tạ Ngọc Thành	MLops , dùng dvc và mlflow để quản lí data và model	- Thiết lập dvc để quản lí dữ liệu và phiên bản model - Triển khai MLflow để theo dõi quá trình huấn luyện - Quản lí model Registry và tích hợp auto training cho hệ thống